TALLER DEPURACIÓN DE CÓDIGO CON IA EN GOOGLE COLAB

Ivan Dario Urrego Herrera

Compiladores

Oscar Fernando Mahecha Camargo

Corporación Universitaria Remington

1. Análisis de Datos con Pandas

-Código original

```
import pandas as pd

# Cargar un archivo CSV

df = pd.read_csv("ventas.csv")

# Mostrar las primeras 5 filas

print(df.head())
```

-Errores identificados y soluciones:

```
# Cargar un archivo CSV

df = pd.read_csv("ventas.csv")
```

El código intenta leer un archivo CSV llamado ventas.csv que no existe o no está disponible en el entorno de ejecución. El problema ocurre porque Pandas necesita encontrar el archivo indicado para crear el DataFrame. Se debe cargar el archivo CSV correctamente o utilizar datos disponibles dentro del entorno.

-Código corregido:

```
import pandas as pd

# Crear datos de ejemplo
datos = {
    "Producto": ["Laptop", "Mouse", "Teclado"],
    "Ventas": [10, 25, 15]
}

# Crear DataFrame
df = pd.DataFrame(datos)

# Mostrar las primeras 5 filas

print(df.head())
```

2. Función Recursiva Incorrecta

-Código original:

```
def factorial(n):
    return n * factorial(n-1)

print(factorial(5))
```

Errores identificados y soluciones:

```
def factorial(n):
    return n * factorial(n-1)
```

La función no tiene una condición de parada y genera una recursión infinita. Una función recursiva necesita un caso base para detener las llamadas. Se agrega una condición cuando n sea 0 o 1.

-Código corregido:

```
def factorial(n):

    # Condición de parada
    if n == 0 or n == 1:
        return 1

    # Llamada recursiva
    return n * factorial(n-1)


print(factorial(5))
```

3. Gráfico de una Distribución con Matplotlib

-Código original:

```
import numpy as np
import matplotlib.pyplot as plt

datos = np.random.normal(loc=50, scale=15, size=1000)

plt.hist(datos)

plt.show()
```

-Error identificado y soluciones:

No se encontraron errores que afecten la ejecución del código. El programa importa correctamente las librerías necesarias, genera los datos mediante una distribución normal y crea el histograma utilizando Matplotlib. Hay que tener en cuanta, que se logra identificar que el histograma puede mejorar, debido a que no se especifican parámetros como la

cantidad de intervalos (bins), etiquetas de los ejes o título del histograma, lo que puede limitar la interpretación de los resultados.

-Código corregido:

```
import numpy as np
import matplotlib.pyplot as plt

datos = np.random.normal(
    loc=50,
    scale=15,
    size=1000
)

plt.hist(
    datos,
    bins=30
)

plt.xlabel("Valores")
plt.ylabel("Frecuencia")
plt.title("Distribución de datos")

plt.show()
```

4. Web Scraping con BeautifulSoup

-Código original:

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"

response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

print(soup.title.text)
```

-Errores identificados y soluciones:

El código funciona correctamente, pero puede mejorarse agregando validación de la respuesta HTTP, manejo de errores, encabezado User-Agent y verificación de los elementos HTML antes de extraer la información. Estas mejoras permiten evitar fallos durante el proceso de Web Scraping.

-Código corregido:

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"

# Agregar identificación del navegador
headers = {
    "User-Agent": "Mozilla/5.0"
}

try:
    # Realizar solicitud con encabezado y tiempo de espera
    response = requests.get(
        url,
        headers=headers,
        timeout=10
    )
    # Validar respuesta HTTP
    if response.status_code == 200:

        soup = BeautifulSoup(
            response.text,
            "html.parser"
        )
        # Verificar si existe el título
        if soup.title:
            print(soup.title.text)
        else:
            print("La página no contiene título.")
    else:
        print(
            "Error al acceder a la página:",
            response.status_code
        )
except requests.exceptions.RequestException as error:
    print(
        "Error en la conexión:",
        error
    )
```

5. Manipulación de Arrays con NumPy

-Código original

```
import numpy as np

array = np.arange(10)
matriz = array.reshape(3, 3)
print(matriz)
```

-Errores identificados y soluciones:

El código presenta un error porque el arreglo contiene 10 elementos, pero se intenta convertir en una matriz de 3x3, la cual requiere 9 elementos. La solución consiste en ajustar

la cantidad de datos del arreglo o modificar las dimensiones de la matriz para que coincidan correctamente.

-Código corregido:

```
import numpy as np

array = np.arange(9)

matriz = array.reshape(3, 3)

print(matriz)
```

6. Uso de Hilos con threading

-Código original:

```
import threading

contador = 0

def incrementar():
    global contador
    for _ in range(1000):
        contador += 1


t1 = threading.Thread(target=incrementar)
t2 = threading.Thread(target=incrementar)


t1.start()
t2.start()

t1.join()
t2.join()


print("Valor final del contador:", contador)
```

-Errores identificados y soluciones:

El código presenta un problema de concurrencia porque varios hilos modifican la misma variable (contador) al mismo tiempo sin ningún mecanismo de sincronización. Esto puede generar resultados inconsistentes debido a que un hilo puede sobrescribir los cambios realizados por otro. La solución consiste en utilizar un bloqueo (Lock) para controlar el acceso a la variable compartida y asegurar que cada hilo modifique el valor de manera ordenada.

-Código corregido:}

```python
import threading

contador = 0
# Crear bloqueo para controlar el acceso a la variable compartida
bloqueo = threading.Lock()
def incrementar():
    global contador
    for _ in range(1000):
        # Evitar que varios hilos modifiquen la variable al mismo tiempo
        with bloqueo:
            contador += 1
# Crear hilos
t1 = threading.Thread(target=incrementar)
t2 = threading.Thread(target=incrementar)
# ejecutar hilos
t1.start()
t2.start()
# Esperar a que finalicen
t1.join()
t2.join()

print("Valor final del contador:", contador)
```

7. API con requests

-Código original:

```python
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

response = requests.get(url)

print(response.json())
```

-Errores identificados y soluciones:

El código funciona correctamente cuando la API responde adecuadamente, pero presenta una falta de control de errores. No verifica si la solicitud fue exitosa antes de intentar obtener la información, por lo que podría generar fallos si la API devuelve un error, como problemas de conexión o códigos HTTP diferentes a 200.

-Código corregido:

```python
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

response = requests.get(url)

# Validar respuesta HTTP
if response.status_code == 200:

    datos = response.json()

    print(datos)

else:

    print(
        "Error HTTP:",
        response.status_code
    )
```

8. Clase con Herencia en Python

-Código original:

```python
class Animal:

    def __init__(self, nombre):
        self.nombre = nombre


class Perro(Animal):

    def __init__(self, nombre, raza):
        self.raza = raza


    def ladrar(self):
        print(f"{self.nombre} está ladrando.")


mi_perro = Perro("Firulais", "Labrador")

mi_perro.ladrar()
```

-Errores identificados y soluciones:

El código presenta un error porque la clase Perro no inicializa correctamente la clase padre Animal, por lo que el atributo nombre no se crea. La solución es utilizar super().__init__(nombre) para heredar correctamente los atributos de la clase base.

-Código corregido:

```python
class Animal:

    def __init__(self, nombre):
        self.nombre = nombre


class Perro(Animal):

    def __init__(self, nombre, raza):
        # Inicializar la clase padre
        super().__init__(nombre)

        self.raza = raza


    def ladrar(self):
        print(f"{self.nombre} está ladrando.")


mi_perro = Perro("Firulais", "Labrador")

mi_perro.ladrar()
```

9. Conexión a Base de Datos SQLite

-Código original:

```python
import sqlite3


conn = sqlite3.connect("database.db")
cursor = conn.cursor()

cursor.execute(
    "CREATE TABLE IF NOT EXISTS usuarios (id INTEGER PRIMARY KEY, nombre TEXT)"
)
cursor.execute(
    "INSERT INTO usuarios (nombre) VALUES ('Juan')"
)

conn.close()
```

-Errores identificados y soluciones:

El código presenta un error porque no confirma los cambios realizados en la base de datos antes de cerrar la conexión. Al realizar la inserción del registro, se debe ejecutar commit() para guardar la información de manera permanente. La solución es agregar conn.commit() antes de cerrar la conexión.

-Código corregido:

```
import sqlite3


conn = sqlite3.connect("database.db")

cursor = conn.cursor()


cursor.execute(
    "CREATE TABLE IF NOT EXISTS usuarios (id INTEGER PRIMARY KEY, nombre TEXT)"
)


cursor.execute(
    "INSERT INTO usuarios (nombre) VALUES ('Juan')"
)


# Guardar los cambios realizados en la base de datos
conn.commit()


conn.close()
```

10. Algoritmo Bubble Sort

-Código original:

```
def bubble_sort(lista):

    n = len(lista)

    for i in range(n):

        for j in range(n-1):

            if lista[j] > lista[j+1]:

                lista[j], lista[j+1] = lista[j+1], lista[j]


numeros = [5, 3, 8, 1, 2]

bubble_sort(numeros)

print(numeros)
```

-Errores identificados y soluciones:

El código presenta un error lógico en el ciclo interno del algoritmo, ya que siempre realiza comparaciones hasta n-1 elementos sin tener en cuenta los valores que ya fueron ordenados en cada pasada. Esto hace que el algoritmo realice comparaciones innecesarias y disminuya su eficiencia. La solución consiste en modificar el rango del segundo ciclo

utilizando n-i-1, permitiendo que en cada iteración se reduzcan las comparaciones y el ordenamiento se realice correctamente.

-Código corregido:

```
def bubble_sort(lista):

    n = len(lista)

    for i in range(n):

        for j in range(0, n - i - 1):

            if lista[j] > lista[j + 1]:

                lista[j], lista[j + 1] = lista[j + 1], lista[j]


numeros = [5, 3, 8, 1, 2]

bubble_sort(numeros)

print(numeros)
```